SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Service Placement through Hierarchical Scheduling at the Edge

Matthew David Humphreys

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Service Placement through Hierarchical Scheduling at the Edge

Service Placement durch Hierarchisches Scheduling am Edge

Author:	Matthew David Humphreys
Examiner:	Prof. Dr.-Ing. Jörg Ott
Supervisor:	M.Sc. Giovanni Bartolomeo
External Supervisor:	Dr. Nitinder Mohan
Submission Date:	01.12.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 01.12.2025 Matthew David Humphreys

Acknowledgments

First and foremost, I wish to express my deepest gratitude to my supervisor, Giovanni Bartolomeo, whose expertise, guidance, and support were crucial in developing this thesis. I could not have hoped for a better supervisor.

To the entire Oakestra team, including former members, I owe my appreciation and respect. I am eternally grateful for the opportunity to work on this esteemed project led by Dr. Nitinder Mohan. This thesis stands on the shoulders of giants.

I thank the chair for Connected Mobility at the Technical University of Munich and Prof. Dr.-Ing. Jörg Ott for the opportunity to work and write my thesis here.

I wish to thank the scouts in Garching and Munich, with whom I always felt accepted, and where my team spirit, creativity, and character were moulded. I am grateful that my absence while working on this thesis was understood, and appreciate everyone who shouldered the load when I struggled to fulfill my responsibilities.

Last, but not least, I thank my parents, friends, and girlfriend, Thanh-Binh, for their support and patience, especially in the more stressful moments in pursuing this degree.

Abstract

Edge computing has emerged as a powerful computing paradigm that enables latency-critical and data-intensive applications. In contrast to cloud computing, edge computing nodes form a heterogeneous topology of constrained devices. This enables the deployment of interlinked latency-critical services closer to each other, and data-intensive services at the network edge. However, it also presents the challenge of efficiently scheduling applications to the optimal compute node. The service placement problem is known to be NP-hard, meaning that narrowing the search space is imperative. This can be achieved through a hierarchical architecture. Aside from latency, other important considerations should be accounted for by the scheduler. Optimization objectives could include resource utilization, carbon intensity, or energy prices. Adding further dimensions to the scheduling decision places an additional computational burden on the scheduler. In this work we present a flexible scheduler for the Oakestra framework, enabling the deployment of varied scheduling algorithms that optimize for different objectives. We leverage this configuration to schedule with respect to different parameters at different levels, enabling multi-objective optimization without incurring additional computational overhead. Six scheduling algorithms are implemented and integrated into the aforementioned scheduler. This thesis compares the efficacy of the presented scheduling algorithms and evaluates their hierarchical permutations. Furthermore, we assess the computational efficiency of the two-stage hierarchical scheduling.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**
1.1 The Service Placement Problem 1
1.2 Contribution 2
1.3 Thesis Structure 3

2 Background **4**
2.1 Problem Statement 4
2.1.1 Abstract Formulation 4
2.1.2 Oakestra Formulation 4
2.2 Related Work 6
2.2.1 Deployment Strategies 6
2.2.2 Scheduling Objectives 6
2.2.3 Metric Collection 7
2.3 Oakestra Architecture 8
2.3.1 Hierarchical Layers 8
2.3.2 Resource Abstractor 11

3 Scheduler Requirements **12**

4 Abstracted Scheduler Implementation **13**
4.1 Architecture 13
4.2 Resource Abstractor 16
4.3 Cluster Orchestrator 16
4.4 Scheduler 17
4.4.1 Task Queue 18
4.4.2 API Endpoints 18
4.4.3 Calculate 18
4.4.4 Scheduling Algorithms 19
4.4.5 Requests 20

5 Scheduling Algorithms **21**
5.1 Latency-Aware Scheduling 21
5.1.1 Latency Categories 21
5.1.2 Filtering Stage Implementation 22
5.2 Implemented Scheduling Algorithms 23
5.3 Caveats and Limitations 25

6 Evaluation **26**
6.1 Experimental Setup 26
6.1.1 Candidate Generation 26
6.1.2 Service Generation 28
6.1.3 Benchmarking 28
6.2 Experimental Results 30
6.2.1 Cluster-Node Configuration 30
6.2.2 Scheduling Algorithms Performance 31
6.2.3 Hierarchical Permutations Evaluation 35

7 Conclusion **39**
7.1 Summary 39
7.2 Limitations 39
7.3 Future Work 40

Abbreviations **41**

List of Figures **42**

List of Tables **43**

Bibliography **44**

1 Introduction

In the decades following the inception of edge computing, its justification has only grown in strength. As the number of Internet of Things (IoT) devices is expected to swell to 24.1 billion in 2030 [1], so will the amount of data that requires generating, transporting, and processing. The proliferation of IoT devices is compounded by the emergence of data-intensive applications such as smart cities, federated learning, and Connected Robotics and Autonomous Systems (CRAS) [2, 3, 4].

Simultaneously with the rising data-intensive demands of current technological trends, latency-critical applications, such as AR/VR, e-health, and Connected Autonomous Vehicles (CAV) [5, 6, 3], are seeing wider adoption.

Many of the above applications are considered driving trends for the formulation of the 6G standard [3], encompassing the push for the efficient transportation of large volumes of data at ultra-low latencies.

Cloud computing has presented itself as an adequate and cost-effective solution to most data-intensive applications. The centralized design of cloud data centers is however not an efficient solution in terms of energy consumption during data transmission [2]. This inefficiency will materialize in the following ways. First, the ingress and egress traffic costs of data-intensive applications will be prohibitively high. Second, the network links will be congested by the density of data-intensive applications, resulting in high service-to-service latency. Furthermore, the communication distances incurred by cloud computing induce an untenable round-trip delay for latency-critical services.

These shortcomings are addressed by edge computing, which solves these problems by facilitating the placement of services at the network edge, enabling the processing of data closer to where it is generated or required [7]. Heavily interlinked applications, such as an AR pipeline constructed from multiple microservices, can be placed in close proximity to each other, alleviating network strain.

1.1 The Service Placement Problem

Despite the apparent benefits of edge computing, few real-world demonstrations have materialized [8]. One reason for this is the technical challenges posed by scheduling services to optimal compute nodes.

Edge resources typically consist of constrained devices, with limited compute and memory capacity. As opposed to cloud service providers, where fractions of a small number of machine types are rented out, edge nodes exhibit considerable heterogeneity in terms of capabilities [9]. Furthermore, the distributed nature of edge implies a heterogeneous topology of network links, with some nodes being considerably better connected than others.

These factors mean that scheduling at the edge is non-trivial and distinct from scheduling in the cloud. The Service Placement Problem (SPP) seeks to address the problem of scheduling a service to the optimal node, considering the above complicating factors. The optimal node may be evaluated according to many different objectives. The most prevalent objective is minimizing latency, but maximizing resource utilization, minimizing carbon intensity, or adhering to legal restrictions are also commonplace [7]. The complexity of the problem, and the plethora of possible approaches, mean that the SPP has been extensively discussed in the literature.

1.2 Contribution

We present a novel approach to the SPP, whereby the hierarchy inherent to the Oakestra framework is leveraged to facilitate the deployment of multiple scheduling algorithms at different hierarchical layers. This is enabled by abstracting the existing scheduling and resource management systems. This arrangement allows us to consider multiple scheduling objectives, without incurring additional computational overhead.

To facilitate this, the Oakestra scheduler is rewritten in a more abstract manner. This enables the rapid integration of new scheduling algorithms, as it operates agnostically with respect to the underlying scheduling algorithm and the resources it considers.

In addition, the Oakestra resource management system, the Resource Abstractor (RA), is extended to the cluster control plane. This component was previously responsible for the collection and abstraction of cluster resources for the Root Orchestrator (RO). The RA is extended to collect and abstract node resources for the Cluster Orchestrator (CO), enabling the deployment of the scheduler at the cluster level.

We introduce a latency classification system which defines discrete categories of latency ranges divided by use-case in edge computing.

The abstraction of the scheduling component and the RA is exploited to implement a small number of scheduling algorithms with distinct objectives. These algorithms are compared using a large-scale simulation. In particular, the permutations of these algorithms are evaluated with respect to their speed and efficacy.

1.3 Thesis Structure

This bachelor's thesis begins by outlining the current status of the SPP in the literature, expanding on appropriate taxonomy and delving into related works. Next, the Oakestra framework's architecture and components are presented. The main body of this work will describe the design of the flexible scheduler, the abstraction and expansion of the Oakestra resource management system, the implementation of individual scheduling algorithms, and the integration of these components into Oakestra. The scheduler is evaluated with respect to its efficiency. Furthermore, the individual scheduling algorithms are contrasted against each other, comparing their speed and efficacy. The algorithms' hierarchical permutations, enabled by the abstracted scheduler, are assessed to determine if they can be configured to yield a superior result. Finally, this thesis concludes with some remarks on the scheduler's benefits and by highlighting some assumptions and limitations. An outlook on future work is provided.

2 Background

2.1 Problem Statement

2.1.1 Abstract Formulation

Suppose $W = \{w_1, w_2, ..., w_{|W|}\}$ is the set of worker nodes which are capable of hosting a service. Further $A = \{A_1, A_2, ..., A_n\}$ is a set of n applications which are provisioned by the application provider. Each application A_i consists of a set of services $A_i = \{s_{i,1}, s_{i,2}, ..., s_{i,|A_i|}\}$. We partition the workers into clusters $C = \{C_1, C_2, ..., C_{|C|}\}$, with $\bigcup_{i \in [|C|]} C_i = W$ and $C_i \cap C_j = \emptyset \, \forall i \neq j$. Each cluster comprises a set of workers $C_i = \{w_a, w_b, ...\}$ where $w_a, w_b, ... \in W$.

Each service $s_{i,j}$ has resource demands, such as memory, number of Virtual Central Processing Unit (vCPU)s or any other arbitrary resource. The set $R = \{r_1, r_2, ...r_{|R|}\}$ encompasses the resources we consider. For example, r_1 may denote `cpu`, r_2 `mem`, etc. Each service's resource demand is denoted as $s_{i,j}^{r_1}$, $s_{i,j}^{r_2}$ or $s_{i,j}^{r_x}$, while each node has available resources, denoted as $w_i^{r_1}$, $w_i^{r_2}$ or $w_i^{r_y}$.

Clusters also have available resources, denoted as $C_i^{r_z}$. The available resources of a cluster are equal to the aggregate of all available resources of the nodes within the cluster,

$$C_i^{r_j} = \bigoplus_{w \in C_i} w^{r_j}$$

where $\bigoplus$ denotes the aggregation operator. We first place service j of application i on cluster C_k, such that $s_{i,j}^r \leq C_k^r \, \forall r \in R$. We then place service $s_{i,j}$ on worker $w_p \in C_k$, such that $s_{i,j}^{r_l} \leq w_p^{r_l} \, \forall r_l \in R$.

2.1.2 Oakestra Formulation

We define a cluster as a collection of workers registered with a CO. The workers regularly report their available resources to the CO, which calculates an aggregated view of the available compute resources within the cluster. The CO reports the aggregated resources to the RO.

The application provider issues an Service Level Agreement (SLA) to the RO. The SLA outlines an array of applications, each consisting of an array of services, for deployment in the system. Each service is a dictionary containing the fully qualified service name, the path to the image file, and a set of key-value pairs outlining the resource requirements of the service. Figure 2.1 shows an example SLA file.

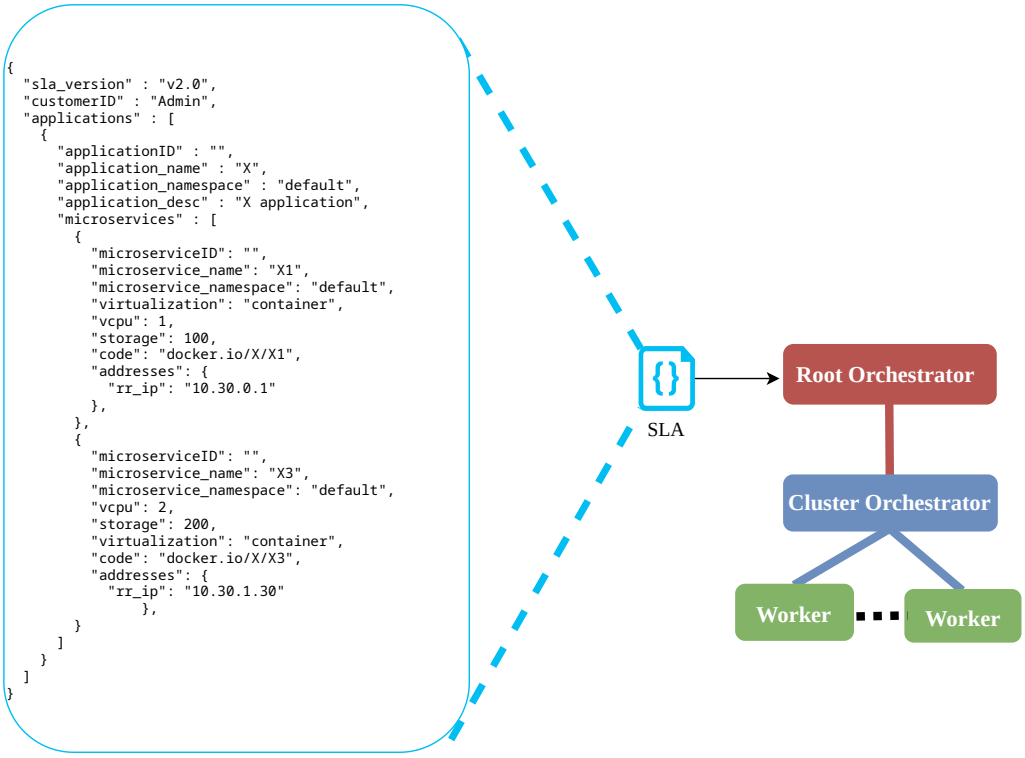

Figure 2.1: An example SLA [10].

The RO evaluates the clusters and their available resources and then sends each service to the most appropriate CO. The CO then schedules the service to the optimal worker node. At each scheduling step, the required resources outlined in the SLA must be less than the available resources reported by the CO or the worker. Provided the minimum resource requirements are met, the scheduler may make further considerations to evaluate the optimal cluster and node.

2.2 Related Work

2.2.1 Deployment Strategies

Salaht et al.'s survey of the SPP identifies four main aspects by which to differentiate deployment strategies [11]:

1. Control plane design: **Centralized** vs **Distributed**.
   The centralized approach enables finding globally optimal solutions. However, distributing the control plane increases the scalability of the system.

2. Placement characteristic: **Online** vs **Offline**.
   Offline scheduling means making all decisions at compile time. This is typically more efficient and allows the consideration of all services to find the optimal placements. Online scheduling offers flexibility in making scheduling decisions at runtime, but is incapable of considering future services.

3. System dynamicity: **Static** vs **Dynamic**.
   Dynamic approaches enable the consideration of changes in the network and compute topology which are commonplace in edge computing networks. This approach is more flexible, but can be less efficient than considering a static system.

4. Mobility support: **Mobile** vs **Non-mobile**.
   Mobile scheduling accounts for changes in user position. This offers flexibility, but increases complexity compared to assuming all users are static.

This thesis addresses decentralized, online, dynamic, and non-mobile placement strategies. The Oakestra framework offers flexibility through online, dynamic scheduling and scalability through a hierarchical distributed approach. Mobility is currently not supported, but future iterations could address this capability gap.

Other works that consider these aspects include Mahmud et al. [12], which prioritises application placement requests according to user expectations, Scoca et al. [13], which calculates a score according to network and compute capabilities, returning the optimal mapping between services and compute nodes, and Tan et al. [14], where a service scheduling algorithm that minimizes job response time is proposed.

2.2.2 Scheduling Objectives

Further important aspects of approaches to the SPP are the considered optimization objectives. The following metrics are commonplace in the literature:

1. **Latency**: minimize the delay between sending a request and receiving a response from the service.
   Latency minimization scheduling algorithms are presented by Souza et al. [15] and Taneja et al. [16].

2. **Resource utilization**: maximize or minimize the utilization of available compute capacity.
   Alsaffar et al. [17] propose an algorithm for balancing the workload across compute devices.

3. **Energy consumption**: minimize the energy consumed in the form of electricity during computation and data transfer.
   Premsankar et al. [18] develop a strategy for the energy-efficient placement of AI applications.

4. **Cost**: minimize the costs incurred by electricity consumption, data link provisioning, and hardware depreciation.
   Rac et al. [19] present a cost-minimizing service placement methodology in the edge-cloud continuum.

5. **Carbon intensity**: minimize the carbon intensity of the required electricity or embodied emissions of the required hardware.
   Asadov et al. [20] propose a spatio-temporal scheduling algorithm for cloud and edge computing that considers operational and embodied emissions.

Notably, many SPP works consider multiple scheduling objectives. Santos et al. [21] demonstrate an Integer Linear Programming (ILP) formulation of the SPP that considers both energy efficiency and latency. An approach that considers carbon intensity and latency is proposed by Do et al. [22]. Trabelsi et al. [23] develop a Mixed-Integer Linear Programming (MILP) algorithm called moDSP that minimizes response time and maximizes resource utilization.

2.2.3 Metric Collection

Many SPP works make strong assumptions regarding the availability and completeness of the data required by the respective scheduling approach. This necessitates reliable approaches to collect relevant metrics.

As latency is a significant factor in edge computing, there are many works on latency metric collection. Haluszczynski proposes a service scheduling approach for the Oakestra framework (previously EdgeIO) that approximates node-to-user latencies by utilizing a Network Coordinate System (NCS). An active probing approach is taken by

Guo et al., who develop pingmesh for large-scale data center latency measurements [24]. This methodology is tailored to cloud data centers, but can be adapted to edge networks.

In order to ascertain the carbon intensity of services deployed at worker nodes, Schmidt et al. present carbond [25]. This is a Linux service that provides carbon information of the underlying hardware to application-level software, providing fine-grained statistics on CO2 emissions for each service instance. A linked metric is provided by ElectricityMaps [26], which provides an API enabling compute devices to determine the carbon intensity of their respective local electricity mix.

2.3 Oakestra Architecture

Kubernetes [27] has emerged as the de facto standard for container orchestration in cloud applications, defining user expectations of an orchestration platform [28]. While Kubernetes' resource profile is too demanding for the constrained devices prevalent at the edge [29], its popularity has inspired many more lightweight distributions. K3s, MicroK8s, K0s, MicroShift, and KubeEdge are all rearchitected for edge computing, but inherit many of the cloud-native design choices made by Kubernetes [30].

Oakestra is a lightweight, hierarchical orchestration framework for edge computing that has been designed from the ground-up with edge in mind [31]. This system endeavors to abstract the underlying components to offer a familiar environment for Kubernetes developers, while meeting the requirements edge imposes. Oakestra organizes compute nodes it manages into clusters, which are administered by a cluster manager, a component of the CO. The clusters are, in turn, administered by the system manager, a component of the RO. For the sake of maintainability, Oakestra is strongly modular.

Throughout this thesis, we refer to the vertical hierarchical divisions of the Oakestra architecture as *orchestration planes*. Each orchestration plane consists of loosely coupled *components* which perform core orchestration functions. The components are deployed as Docker containers and communicate over HTTP facilitated by Docker Compose networking [32]. These components are constructed from tightly coupled logical units which we refer to as *modules*. Modules are implemented as Go packages [33] or Python modules [34] and communicate over shared memory. We use the terms worker and node interchangeably.

2.3.1 Hierarchical Layers

The application provider registers a service in the form of an SLA with the Oakestra front-end and commissions the deployment of a number of service instances. The RO accepts the service descriptor and propagates it to the appropriate cluster, as per the

scheduler. The CO then directs the service to the most suitable worker node, where it is deployed.

Root Orchestrator

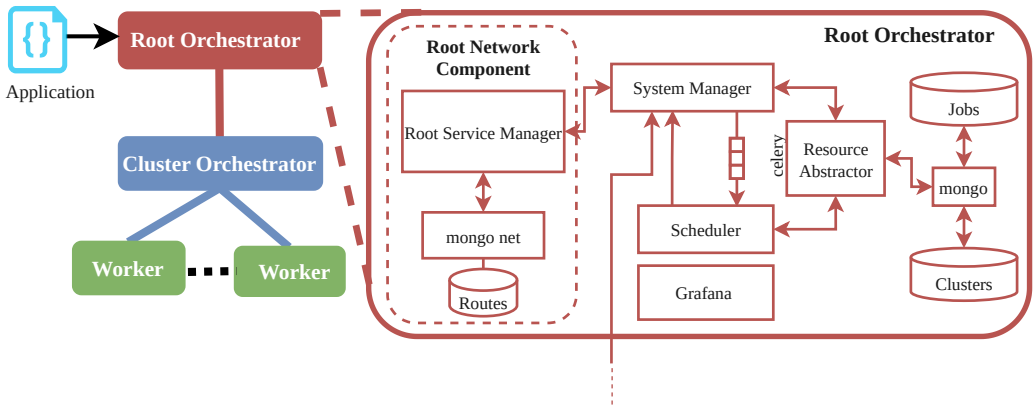

Figure 2.2: Architecture of the root orchestrator [10].

The RO forms the foundation of the Oakestra system. It consists of the following components as shown in Figure 2.2.

The **System Manager** administers the clusters and is the focal point for user interactions. It accepts service descriptors from the application provider, consults the scheduler, propagates the service to the appropriate cluster, and then informs the RA. The clusters report the available resources to the system manager, which forwards these reports to the RA.

The **Scheduler** receives scheduling requests from the system manager. This component queries the RA for the available clusters and their resources, computes the optimal cluster placement, and responds to the system manager with the appropriate cluster.

The **Resource Abstractor** accepts payloads from the system manager that describe the available cluster resources and the deployed jobs. It stores this information in a MongoDB database [35]. The RA provides API endpoints to query the cluster resources and deployed jobs.

The **Root Service Manager** is the networking component of the RO. It provides semantic and instance IP namespaces for each service and the clusters' subnetworks.

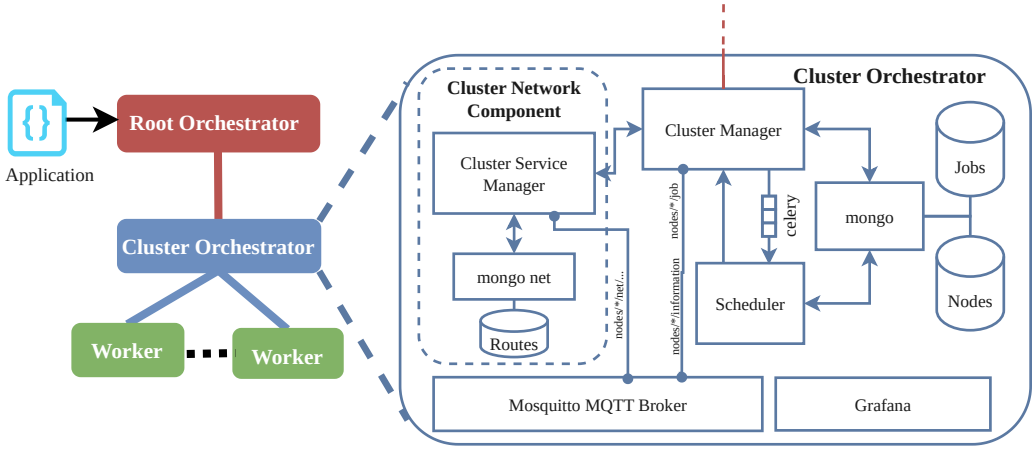

Figure 2.3: Architecture of the cluster orchestrator [10].

Cluster Orchestrator

The CO is the vertically central orchestration plane. As outlined in Figure 2.3, it consists of the following components.

The **Cluster Manager** administers the workers and aggregates their available resources, which it reports to the system manager. It accepts service instances from the system manager, consults the scheduler, propagates the service instance to the appropriate worker, and records its deployment to the MongoDB database. The workers report the available resources, which the cluster manager records in the database.

The **Scheduler** receives scheduling requests from the cluster manager. This component queries the database for the available workers and their resources, computes the optimal worker placement, and responds to the cluster manager with the appropriate node.

The **Cluster Service Manager** is the networking component of the CO. It provides semantic and instance IPs to each service and worker.

Worker Nodes

The worker node is the component responsible for executing workloads. As shown in Figure 2.4 it consists of two components.

The **NodeEngine** manages the instance deployment based on installed runtimes. It reports on available resources and the state of the service instances to the cluster manager.

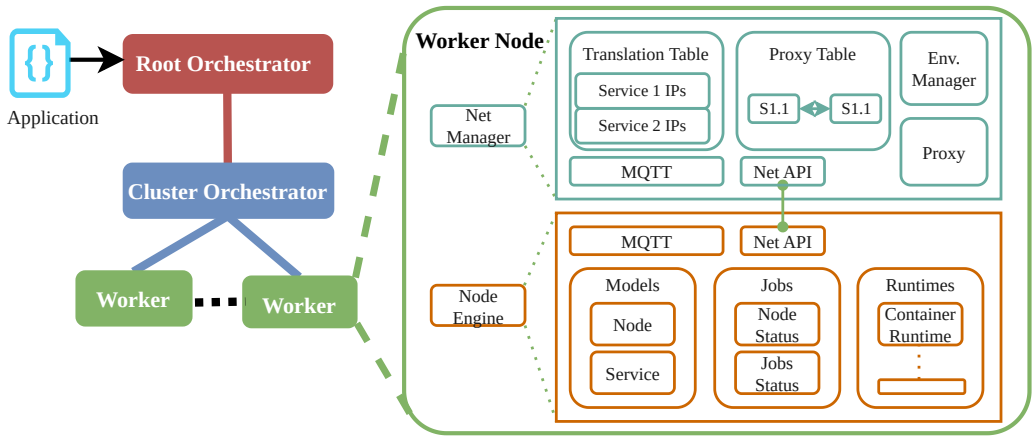

Figure 2.4: Architecture of the worker node [10].

The **NetManager** facilitates the communication between services deployed across multiple nodes. It receives instance and resolves service IPs via the cluster service manager.

2.3.2 Resource Abstractor

The RA is the resource management system of the RO. It acts as an intermediary between the MongoDB database at one end and the system manager and scheduling module at the other end. The database stores the aggregated cluster resources reported by the CO. Moreover, it stores the service descriptors and the location of deployed service instances. The RA ensures data consistency for the read and write operations to the database. Furthermore, it enables the extensions system [36].

The extensions system enables the modification of the behavior of the Oakestra framework so that it can be tailored to specific needs, without incurring an undue performance overhead. It comprises three subsystems: Addons, Hooks, and Custom Resources. Addons permit the implementation of additional, or the replacing of, RO components. Hooks allow components to receive notifications for lifecycle events within the system. In particular, components can be informed if a service is created, updated, or deleted. Custom Resources dynamically extend the RA API to store and query structured data.

3 Scheduler Requirements

In the design process of the scheduler, and through reviewing the existing scheduling components, we arrived at the following requirements for the scheduler:

1. **Scalable:** As the SPP is considered to be NP-hard, the scheduler may require a considerable amount of time to calculate the result of a scheduling request [11]. However, the scheduler must be able to accept a large number of requests simultaneously, as the application provider may issue many service descriptors at once.

2. **Modular:** Modularity is one of the core tenets of the Oakestra framework [31]. As such, all components must be self-contained and interface with other components over exposed APIs. This scheduler is intended to replace the existing scheduling components for the RO and the CO, and therefore must provide the same API endpoints to receive scheduling requests. The scheduler should interface with the existing endpoints at the RA to query the available resources.

3. **Resource Agnostic:** In order to facilitate a diverse array of scheduling algorithms, which may consider different resources, the scheduler must operate without any assumptions about the resource types provided by the RA.

4. **High Availability:** The scheduler must be fault-tolerant and recover from internal errors or improperly formatted data. Should an error, occur it must be caught and propagated via the API.

5. **Lightweight:** A lightweight design is a further core tenet of the Oakestra framework [10]. As such, the scheduler must not be a bottleneck for deployment operations.

6. **Maintainable:** In order to facilitate future enhancements, or to rapidly address potential defects, the scheduler must be implemented in a maintainable manner. Therefore, an organized architecture, thorough documentation, and unit tests for central operations are required.

4 Abstracted Scheduler Implementation

Throughout this thesis, we refer to the *scheduler* as the component responsible for accepting an SLA from the system or cluster manager and returning a placement candidate. We refer to the *scheduling algorithm* as the module within the scheduler responsible for evaluating the candidates with respect to the service descriptor. A placement *candidate* is a valid scheduling target, either a node or a cluster.

We abstract the scheduler by implementing all operations without any assumptions about the resource structure of the placement candidates or the service descriptors. We make this decision for two reasons: First, it allows us to deploy the same scheduling component at both orchestration planes, as the scheduler is indifferent to the structural differences between nodes and clusters. In particular, it is capable of considering node resources and aggregated cluster resources. In using the same component at both orchestration planes we increase the maintainability and modularity of the overall system. Second, by discarding any assumptions on the resource profile of a candidate, we can rapidly implement new scheduling algorithms capable of considering different resources in a novel manner.

The scheduler was abstracted so that the same component may be used by the RO and the CO. In this process the scheduler, the RA, and the CO were modified. While the changes to the scheduler amount to a complete overhaul, the changes to the other components were kept to a minimum.

4.1 Architecture

Figure 4.1 shows how the orchestration planes were rearchitected for the abstracted scheduler. The RA component was extracted from the RO and is deployed as two instances across both orchestration planes. The scheduling module was also removed from the RO. Mirroring the RA, the scheduler is deployed once at the root orchestration plane and once at the cluster orchestration plane. The scheduling module and the MongoDB database are removed from the CO.

As a result, the RO and CO are significantly reduced in complexity. The root and the cluster orchestration planes no longer have a unique resource management and scheduling system each, but now both utilize separate instances of the same component.

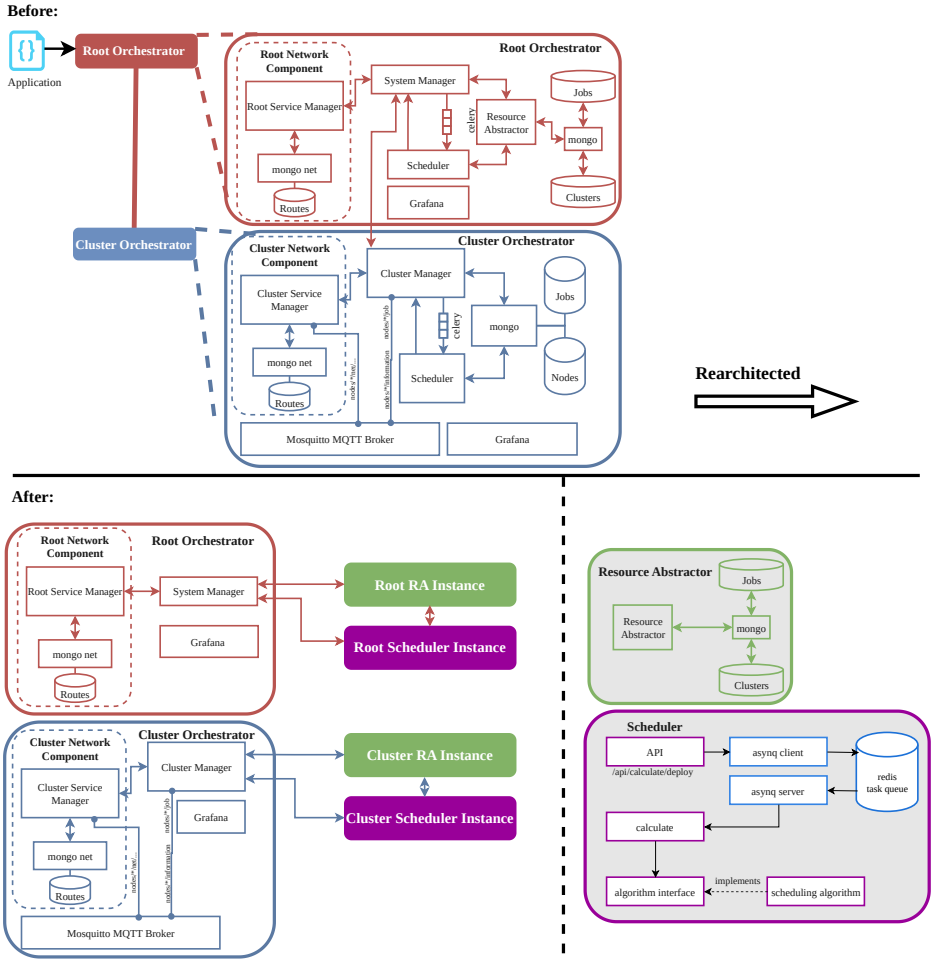

Figure 4.1: Rearchitecting the orchestration plane.

Table 4.1: Canonical and Non-canonical Resources.

Resource	Type	Aggregation Scheme
Canonical		
cpu_percent	float	average
vcpus	integer	total
memory_percent	float	average
memory	integer	total
vram	float	total
vram_percent	float	average
gpu_temp	float	average
gpu_drivers	string list	flatten
gpu_percent	float	average
vgpus	integer	total
virtualization	string	append
supported_addons	string list	flatten
Non-Canonical (Examples)		
carbon_intensity	float	default (total)
electricity_price	float	default (total)
location	string	default (append)
latency	map: string -> integer	default (none)

4.2 Resource Abstractor

We extend the RA from the root orchestration plane to the cluster orchestration plane. This decision is made as the scheduler interfaces with the RA. Therefore, deploying the scheduler for the CO also necessitates the deployment of the RA at the cluster. This rearchitecting is further justified as using the same component in both hierarchical layers increases the maintainability of the system and enables the extensions system for the CO.

Previously, a static list of keys was used to collect and serve cluster resources for the RO. In extending the RA to the cluster, we observe the challenge presented by not having prior information on a node's resource profile, due to device heterogeneity. Moreover, we wish to facilitate an abstract scheduler by permitting the collection of any resource.

We therefore introduce the distinction between "canonical resources" and "non-canonical resources", as shown in Table 4.1. The former are resources we always expect a candidate to possess. These are always collected and served by the RA. The latter optionally supplement the canonical resources. These are always collected, but only served when explicitly requested.

This change enables the deployment of the RA at the cluster control plane. The heterogeneity of node resources is managed by indiscriminately accepting all values provided by the node and collecting them as non-canonical resources.

We modify the behavior of the API endpoint that accepts GET requests from an orchestrator, so that the query may contain a list of interested resources. The accounted resources are projected onto the union of the canonical and interested resources before being returned as JSON. This change enhances the flexibility of the RA, without polluting the result with superfluous information.

4.3 Cluster Orchestrator

The deployment of the RA at the cluster control plane requires further modifications to the CO. Previously, node data was accepted according to the MQTT protocol [37] using a fixed schema. The CO was responsible for the storage of this data using a MongoDB database [35] and for the scheduling of service instances to worker nodes. We refactor both of these tasks into dedicated components.

In every case where the CO would interface with the MongoDB database, we instead utilize the API provided by the RA. To this end, the node data packets are no longer manually traversed for known keys. Instead, the entire payload is forwarded to the RA, so that both canonical and non-canonical resources may be collected.

Delegating the resource management to the RA enables interfacing with the abstracted scheduler. We remove the integrated scheduling module and instead forward the service instances to the scheduling component. In turn, the scheduler interfaces with the RA to retrieve information on the available worker nodes and their resources.

A further task the CO performs is the aggregation of resource statistics for the RO. This enables an abstracted view of the available resources in a cluster to the RO. For each resource type we iterate over all workers in order to calculate an aggregated value. The aggregation of canonical resources is performed by applying a fixed aggregation scheme to each resource type. Due to the indeterminate nature of the non-canonical resources, we cannot provide a straightforward strategy to aggregate each resource. Therefore, an abstract aggregation scheme is introduced, which operates according to the value type of the resource statistic in question, as outlined in Table 4.2.

Table 4.2: Abstract aggregation scheme.

Resource Value Type	Aggregation Scheme
integer	summate
float	summate
string	append to list
string list	flatten to list

4.4 Scheduler

Prior to introducing the abstracted scheduler, its tasks were performed by modules within the RO and CO. This functionality has been extracted to an independent component, which receives scheduling requests, containing service descriptors, from the orchestrators. It then queries the RA to ascertain the placement candidates and their available resources. Finally, the scheduler replies to the orchestrator with the identifier of the chosen placement candidate or an error describing the scheduling failure.

This component was written in the Go programming language [38]. The decision to transition from Python to Go was made for the following reasons: Go offers type safety, significantly higher performance for most applications [39], and support for interface-based polymorphism. This contributes to our lightweight and maintainable requirements. Figure 4.2 shows the architecture of the scheduler. In the following, we explain the function of every module.

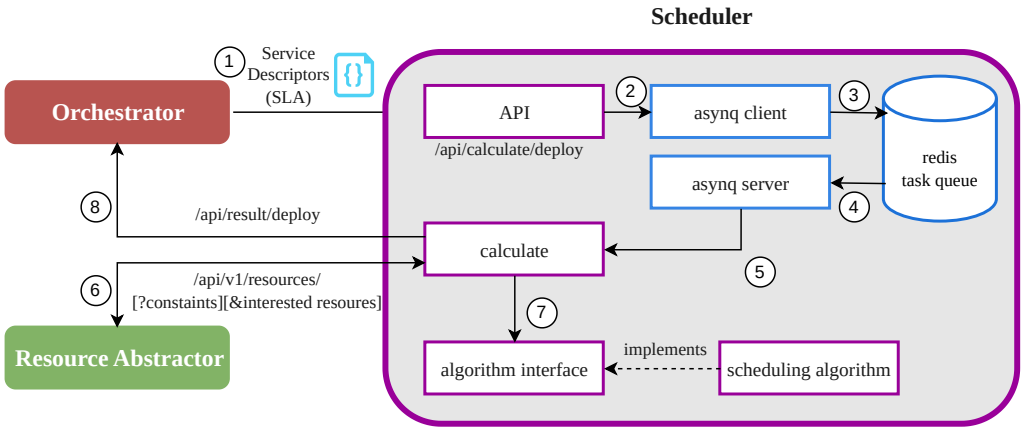

Figure 4.2: Architecture of the scheduling component.

4.4.1 Task Queue

To meet the scalability requirement, the scheduler must be capable of accepting incoming requests while it is still processing others. This requirement could be met by processing the requests in parallel, spawning a new routine for each scheduling task. This is, however, impractical as it could lead to race conditions, yielding inconsistent results such as over-provisioning of resources.

We address this by implementing a task queue system with Asynq, as it is well-maintained and lightweight [40]. An Asynq client enqueues incoming tasks into a queue hosted by a Redis database [41], from which an Asynq server retrieves them for processing.

4.4.2 API Endpoints

The scheduler provides two API endpoints through which other components may communicate with it. The first endpoint accepts a GET request and returns an HTTP 200 status code, serving to ascertain the reachability of the component. The other accepts a POST request with a payload carrying the descriptor of the service to be deployed. This descriptor is then forwarded to the Asynq client and enqueued.

4.4.3 Calculate

We observe that the service descriptor, or SLA, and a placement candidate encapsulate the same information. We therefore unify both concepts into a single data type called

the `ResourceList`. This type consists of key-value pairs, whereby each key refers to an available resource on the candidate side or a required resource on the SLA side.

The `ResourceList` is realized as an interface, which may be implemented by a scheduling algorithm. We use this abstract interface as a placeholder for most scheduler operations, without requiring prior knowledge of the underlying data.

We introduce a further interface called `Algorithm`, which represents an underlying scheduling algorithm. Each `Algorithm` requires an associated `ResourceList`, and is as such parameterized with one, utilizing Go generics. This interface provides functions to retrieve the correlated `ResourceList` and a `calculate` function, central to the scheduler's operations. This function accepts a service and a list of candidates, both of the appropriate `ResourceList` type, and returns the selected placement candidate.

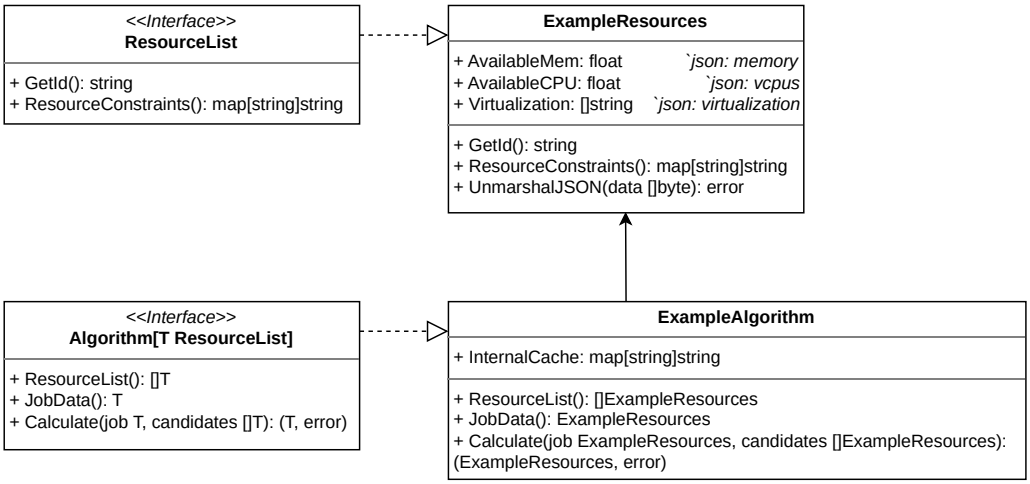

Figure 4.3: Example `ResourceList` and `Algorithm` implementation.

4.4.4 Scheduling Algorithms

A new scheduling algorithm is introduced by implementing the `ResourceList` and `Algorithm` interfaces as in Figure 4.3.

The `ResourceList` structure is populated with interested resources. These are resources the scheduling algorithm might consider. The keys are annotated with JSON tags to enable the serialization and deserialization for communication over HTTP.

`Algorithm` implementations provide functions to retrieve the associated `ResourceList` type as objects. This enables the capturing of `ResourceList` data from the orchestrator, for a service, or from the RA, for the candidates. The `calculate` function processes the

available candidates in two stages. First, the candidates are filtered out in a single pass. Here any candidate that does not meet the minimum requirements, such as vCPU, memory, or virtualization technology, is excluded from further consideration. The remaining candidates are then ranked according to a scoring function. This function could sort candidates according to their available memory and processing resources, their carbon intensity, or their average latency to other nodes. The optimal candidate is then returned.

4.4.5 Requests

The scheduler communicates with two further components: the orchestrator and the RA. Both of these components provide API endpoints for the scheduler.

We query the RA for information on the available candidates and their resources. To perform this request, we utilize the `ResourceList` to ascertain the interested resources by traversing the keys and creating a list of resource names. This list is encoded into the query string of the API request to the RA. The response is a JSON payload containing a list of dictionaries, each representing a placement candidate. The results contain both the canonical resources and the queried resources. This payload is unmarshaled into the provided slice of `ResourceList` objects.

The scheduling result is communicated to the RO or CO. As there may be a significant delay between receiving the scheduling request and completing the calculation, the scheduler does not directly reply to the HTTP request with a response. Instead, it sends a POST request on completing the calculation. To allow the orchestrator to track the scheduled service, the request includes the identifier of the service, as well as the identifier of the chosen candidate. In the event of a scheduling failure, when no suitable candidate can be found, the payload carries a status field indicating the error.

5 Scheduling Algorithms

Six scheduling algorithms are integrated into the scheduler, where each algorithm considers a distinct scheduling objective. These implementations serve to demonstrate the flexibility provided by the scheduler architecture and to highlight the benefits of scheduling with respect to different objectives at each orchestration layer.

5.1 Latency-Aware Scheduling

Due to the significance of latency in edge computing, we implement six latency-aware scheduling algorithms.

As microservices continue to increase in prevalence [42], the fraction of Service-to-Service (S2S) requests outpaces that of Service-to-User (S2U) requests. Research shows that the proportion of internal API requests of total API requests can be as high as 99.7% in a cloud-native application [43]. Furthermore, measuring S2S latency is significantly simpler than S2U latency, as we do not have to account for the mobility and multiplicity of end users. The implemented algorithms therefore consider the S2S latency of interdependent services.

5.1.1 Latency Categories

Table 5.1: Proposed latency categories.

Score	Use Case	Latency Range
0	Instant (Same Device)	0 ms
1	Motion-to-Photon	0 - 20 ms
2	Teleoperation	21 - 50 ms
3	Perceivable	51 - 100 ms
4	Flow-of-Thought	101 - 1000 ms
5	Non-Interactive	> 1 s

We propose five latency ranges, excluding the same-device case, categorized according to use case as outlined in Table 5.1. The latency ranges are defined in this

manner as the application provider is unlikely to specify a precise latency requirement in milliseconds, instead specifying the latency requirement according to the nature of the service relationship. Furthermore, the latencies between two nodes are in constant flux as traffic routing continuously adapts [44]. A fine-grained latency demand could therefore lead to frequent SLA violations, potentially incurring monetary costs.

The latency categories are defined as follows:

1. **Motion-to-Photon ($\leq$ 20 ms):** The latency requirements for a seamless AR/VR application [45].
2. **Teleoperation ($\leq$ 50 ms):** The latency requirements for remote surgery, applicable to other forms of latency-critical remote control [46].
3. **Perceivable ($\leq$ 100 ms):** The latency requirements for interactive applications such as video games or Unmanned Aerial Vehicle (UAV) piloting [47, 48].
4. **Flow-of-Thought ($\leq$ 1 s):** The latency requirements for a search engine or other user interface [49].
5. **Non-Interactive (> 1 s):** No strict latency requirements for non-interactive applications.

We assign each latency range a score from zero to five, where a lower score indicates a better performance. In the event that interdependent services are scheduled to the same device, we grant the latency relationship a score of zero.

5.1.2 Filtering Stage Implementation

The latency-aware scheduling algorithms verify the latency requirements in the filtering stage. The SLA defines the latency requirements by mapping service dependencies to latency score demands. These mappings are expected to be symmetric as the scheduling algorithms do not consider a directed latency graph. The RA has information on the inter-candidate latency at the corresponding orchestration plane in the form of a complete symmetric latency matrix. Each candidate possesses a structurally identical entry to the SLA in the form of a map of every outgoing candidate connection to its corresponding latency score. Figure 5.1 demonstrates how the scheduler would find the appropriate placement candidate.

The scheduling algorithm ascertains if the incoming SLA contains any job dependencies. Should this be the case, the internal cache is used to find the candidate to which the corresponding service has been scheduled. If all the corresponding services have not yet been deployed, the latency requirement is considered met. Otherwise, the

scheduling algorithm will exempt all candidates whose latency score to the candidate on which the dependent service in question is deployed is higher than the SLA demands from further consideration. Should no candidate meet these requirements, the scheduling fails with an error. The latency graph is not considered any further by a scheduling algorithm except Worst and Best Latency Fit.

As shown in Figure 5.2, the filtering stage of the scheduling algorithms additionally ensures the considered candidates support the correct virtualization technology, have sufficient vCPUs, and meet the minimum available memory requirement.

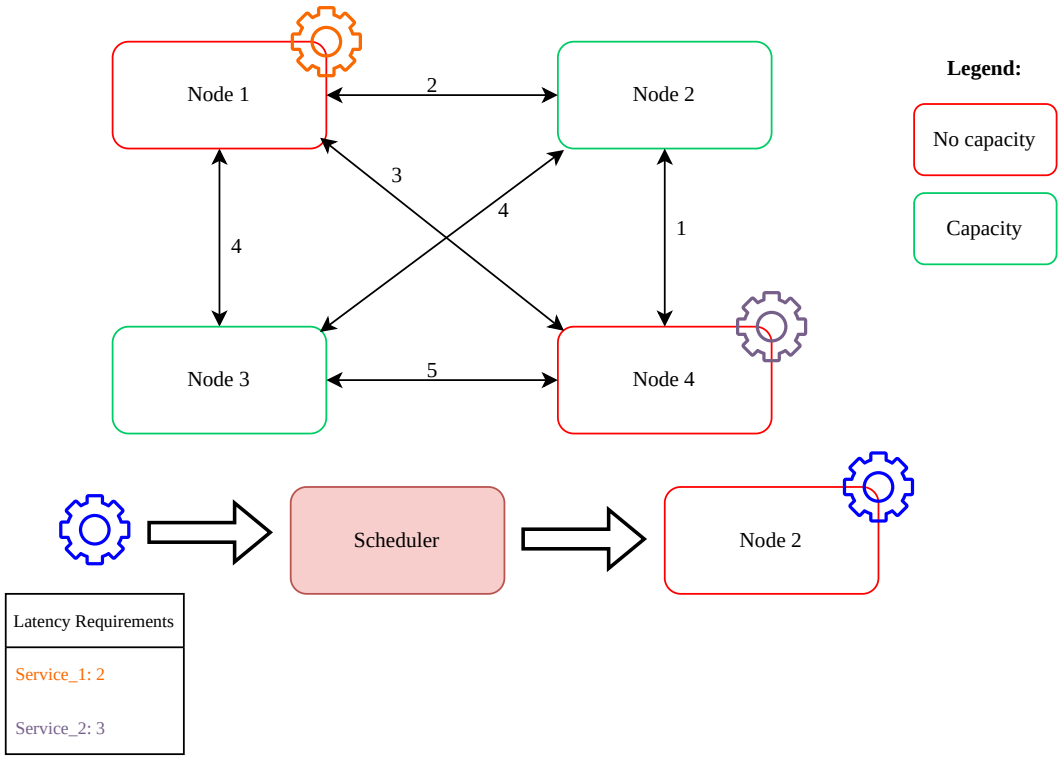

Figure 5.1: Latency-aware scheduling example.

5.2 Implemented Scheduling Algorithms

The following latency-aware scheduling algorithms are implemented, where the "-Fit" suffix refers to the above filtering stage:

1. **BestMemoryFit** considers the scheduling objective of resource utilization. It

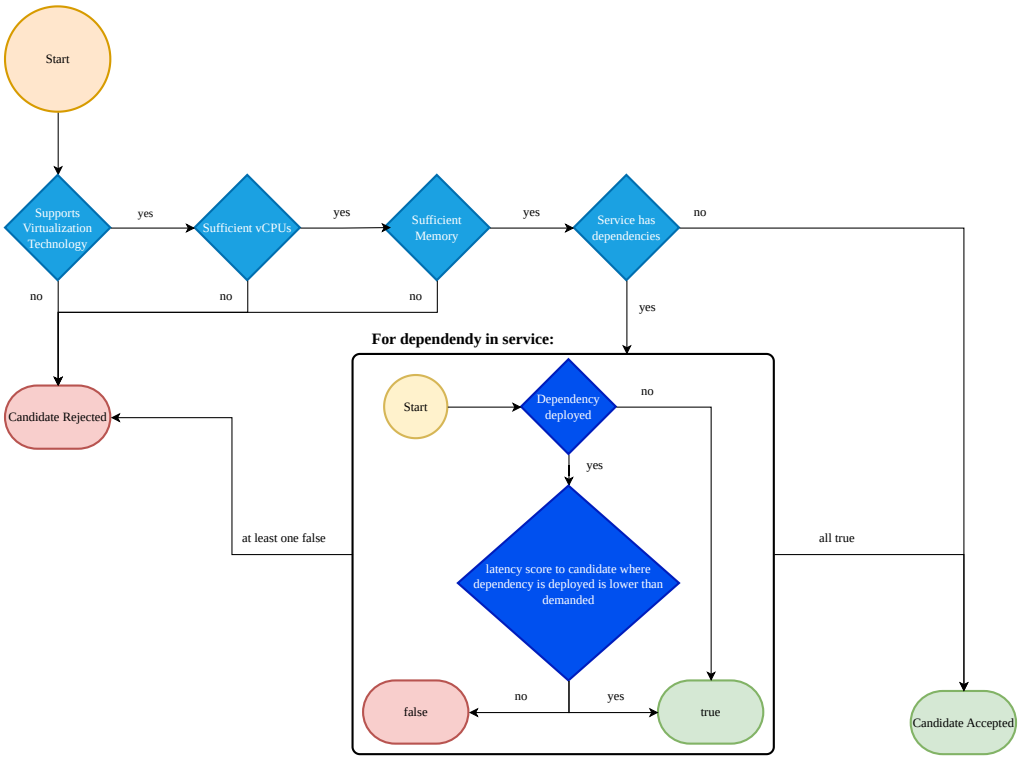

Figure 5.2: Latency-aware candidate filtering process.

attempts to maximize the potential service density on a given candidate.

2. **WorstMemoryFit** considers the scheduling objective of resource utilization. It attempts to evenly distribute the services.
3. **BestLatencyFit** considers the scheduling objective of latency. It attempts to reserve well-connected candidates for services with stringent latency requirements.
4. **WorstLatencyFit** considers the scheduling objective of latency. It attempts to deploy services to well-connected candidates whenever possible.
5. **LowestCarbonFit** considers the scheduling objective of carbon intensity. It attempts to prioritize candidates with a lower carbon intensity.
6. **RandomFit** serves as a baseline to evaluate other algorithms against.

The implemented scheduling algorithms are accessible at a public archived repository [50].

5.3 Caveats and Limitations

The scheduling algorithms assume complete information on the state of the latency matrix and the carbon intensity of the placement candidates. Furthermore, the undeployment or termination of services is not taken into account by the scheduler. Addressing these issues is beyond the scope of this thesis.

6 Evaluation

We evaluate the scheduler using the benchmarking functionality provided by the Go testing standard library package. This enables us to inspect the scheduling component in isolation. The nodes, clusters, and services are generated and then the results of each hierarchical algorithm pairing are analyzed. As six scheduling algorithms were implemented, this yields $6^2 = 36$ permutations.

6.1 Experimental Setup

```
type LatencyAwareResources struct {
        Id              string
        JobName         string
        AvailableMem    float64
        AvailableCPU    float64
        Virtualization  []string
        Latency         map[string]int
        CarbonIntensity float64
}
```

Figure 6.1: `LatencyAwareResources` used for benchmarking.

6.1.1 Candidate Generation

We generate n clusters, each containing m nodes. The `ResourceList` interface is implemented with the `LatencyAwareResources` as shown in Figure 6.1. This resource structure is valid for all implemented scheduling algorithms, as resources that are irrelevant to the particular scheduling algorithm are ignored by the scheduler.

Nodes

To emulate the device heterogeneity expected in edge computing, we define three types of nodes: small, medium, and large as shown in Figure 6.2. The medium node size is

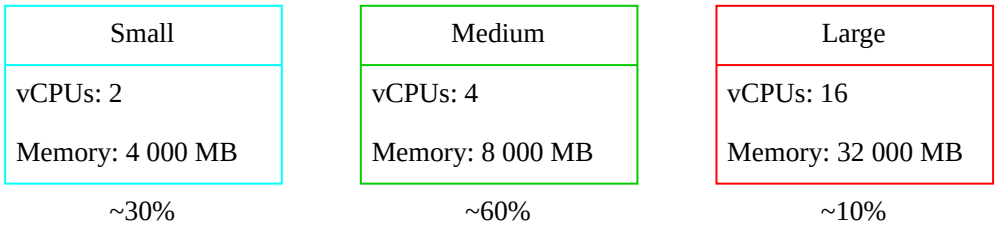

Figure 6.2: Benchmarking node distribution.

the most common, followed by small and then large, where large is significantly less common. Within each cluster, a complete and symmetric latency matrix is defined. The `Latency` map entry of each node contains m entries, with the key being the `Id` of every node within the cluster, and the value being the latency score.

The latency scores between nodes are randomly generated values between one and six, according to the distribution outlined in Table 6.1. The only exception is the node latency score to itself, which is always zero.

Table 6.1: Inter-node latency distribution.

Latency Score	Relative Frequency
1	~20%
2	~30%
3	~30%
4	~10%
5	~5%
6	~5%

The carbon intensity of a node is inherited from the generated cluster, as the majority of the CO2 emissions of a compute node arise during the operational phase [51], which is heavily influenced by the carbon intensity of the consumed electricity. We expect clusters to be geographically confined, and therefore to have a similar electricity mix. Consequently, we vary the carbon intensity of a node by at most $\pm 15\%$ of the cluster carbon intensity.

Clusters

The carbon intensity of the clusters is determined by selecting a random sample from a real-world dataset of the carbon intensity of electricity grids around the globe. This

dataset was collected using the Carbon Aware SDK [52] over multiple years of metric collection.

The aggregation step performed by the CO is replicated by assigning each cluster the total value of the memory and vCPUs of the nodes assigned to it.

The latency matrix is constructed in a similar manner to that of the nodes within a cluster, however it is skewed significantly higher, as shown in Table 6.2.

Table 6.2: Inter-cluster latency distribution.

Latency Score	Relative Frequency
4	~30%
5	~50%
6	~20%

6.1.2 Service Generation

The services are likewise objects of the type `LatencyAwareResources`. Each service has memory and vCPU requirements which are randomly selected from the defined ranges shown in Table 6.3. The random selection is skewed to low numbers in order to simulate many lightweight services and a small number of computationally intensive services.

Services may have latency requirements to other services. We refer to these as service dependencies. Approximately 60% of the services are granted service dependencies. Analogously to the candidate latency relationships, the service latency relationships are always symmetric. This is due to the fact that the implemented scheduling algorithms do not consider the dependency direction. Furthermore, this prevents the order in which the services are dispatched to the scheduler from having an effect.

Table 6.3: Service resource requirements.

Resource	Minimum	Maximum
Memory	500	8000
vCPUs	0	4

6.1.3 Benchmarking

We are interested in the impact of the following variables:

1. The node-cluster configuration.
2. The individual scheduling algorithms.
3. The permutations of the scheduling algorithms.

We ensure statistical significance of the results by running 10000 trials. Within each trial we analyze all six cluster-node configurations `1-45`, `3-15`, `5-9`, `9-5`, `15-3`, `45-1`. According to each cluster-node configuration we generate the candidates and services as outlined above. With the generated candidates and services we iterate over each scheduling algorithm permutation, whereby one is the root scheduler, and the other is the cluster scheduler.

The benchmarking framework provides a dynamic counter `N`, which grows until a statistically significant threshold in terms of time elapsed has been reached. For each algorithm pairing we schedule 30 services. We iterate over all services `N` times.

Each service is first scheduled according to the root scheduler. Upon finding a suitable candidate the service's required resources are deducted from the cluster. The service is then scheduled according to the cluster scheduler, whereupon the required resources are deducted from the node. Upon scheduling all services, the cluster and node resources are reset for the next iteration.

After each iteration we collect the following statistics:

1. The average **time** in milliseconds that elapsed during the execution of the calculate functions. We collect the
   a) Root scheduling time.
   b) Cluster scheduling time.
2. The average service-to-service **latency**. This is collected by iterating over all deployed services, resolving any dependencies, and accumulating the latency scores to the appropriate nodes. In the event that inter-dependent services are deployed across clusters, the inter-cluster latency score is used.
3. The average **carbon intensity** of the nodes or clusters, weighted according to the amount of provisioned memory by each candidate. This weighting is used as an approximation of the utilized compute capacity.
4. The **distribution index**. This is calculated according to Jain's fairness index with respect to provisioned memory.
5. The total number of **scheduling errors**.

Upon completing the scheduling process for all services `N` times, we compute the average results for the elapsed time, latency, carbon intensity, resource distribution, and total errors and record the results.

6.2 Experimental Results

6.2.1 Cluster-Node Configuration

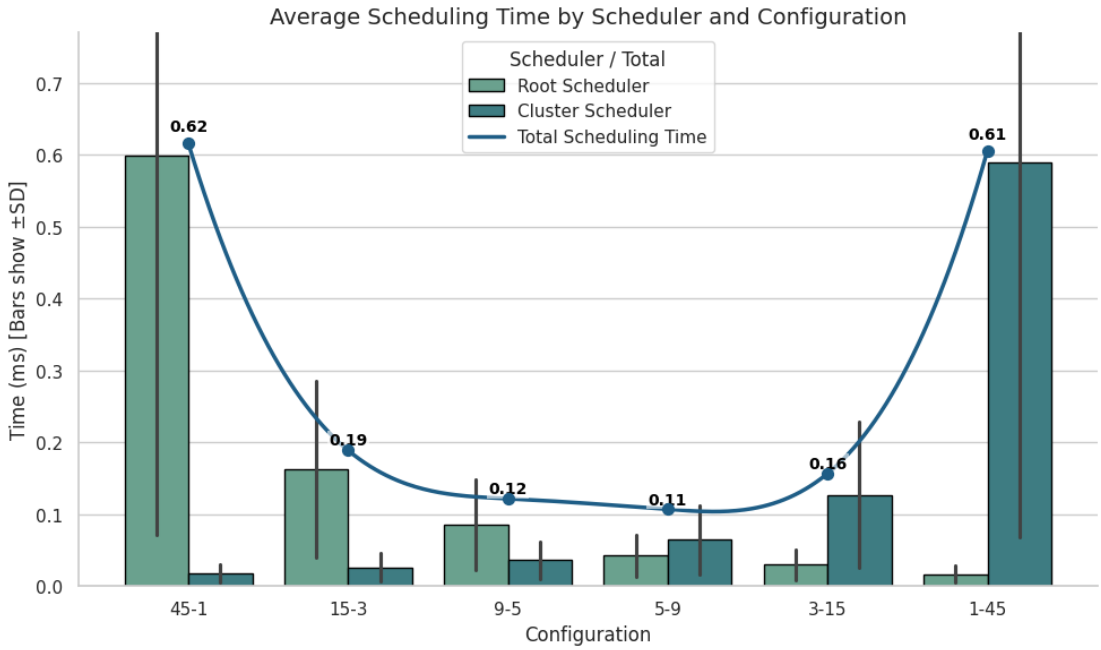

Figure 6.3: Scheduling time by cluster-node configuration.

Figure 6.3 shows the average scheduling time across all scheduling algorithms with respect to the cluster-node configuration. As expected, the root scheduler calculations demand more time when we configure proportionally more clusters, and the cluster scheduler calculations are slower the more workers per cluster we configure.

Furthermore, we observe the total scheduling time is significantly lower, the more even the cluster-node distribution is. This supports Oakestra's design decision to structure scheduling hierarchically [31].

The high variance is likely caused by the profound differences in scheduling time between earlier services and later ones, before the simulation is reset. As the number of services within the system grows, so does the complexity of scheduling further services,

in particular with respect to the resolution of latency requirements.

6.2.2 Scheduling Algorithms Performance

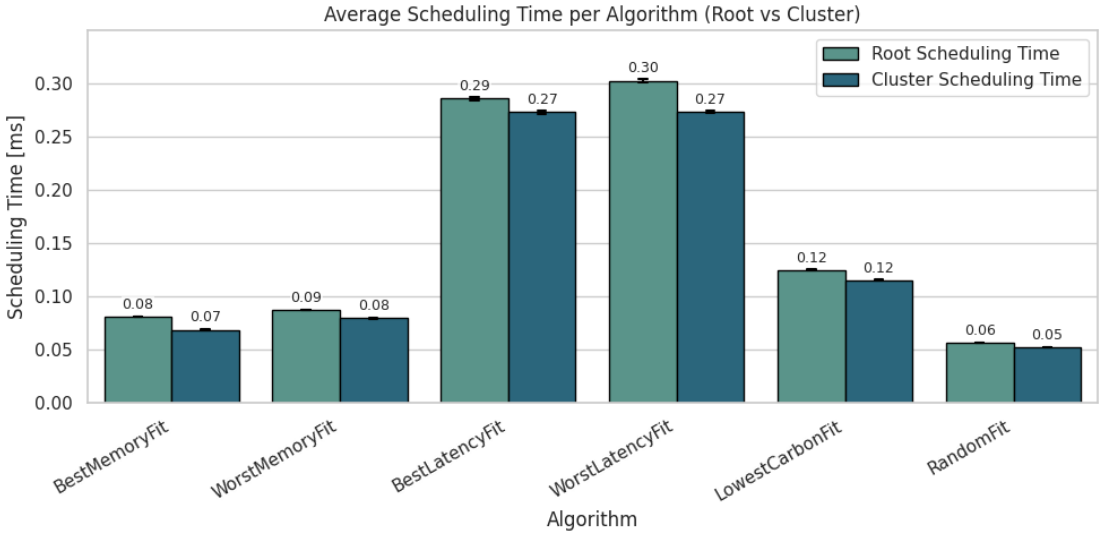

Figure 6.4: Computational intensity of scheduling algorithms.

We evaluate the six outlined scheduling algorithms according to their efficacy and computational intensity. Figure 6.4 shows the scheduling time required by each algorithm at each orchestration plane.

RandomFit performs the best, as it only performs the filtering stage before returning a uniformly selected suitable candidate.

BestMemoryFit and WorstMemoryFit both perform similarly well to each other, as they iterate over every candidate post filtering stage and return the one with the lowest or highest available memory. On average they take respectively 36% and 55% longer than RandomFit.

LowestCarbonFit performs slightly worse than BestMemoryFit and WorstMemoryFit, however it is on average 118% slower than RandomFit. It iterates over all filtered candidates and returns the one with the lowest carbon intensity.

BestLatencyFit and WorstLatencyFit both perform similarly poorly, as they must sum all the outgoing latency values of each node before returning the candidate with the highest or lowest aggregate latency. These algorithms are still significantly slower when caching the latency sum values, each requiring over 400% more time than RandomFit to yield a result.

WorstLatencyFit and BestLatencyFit

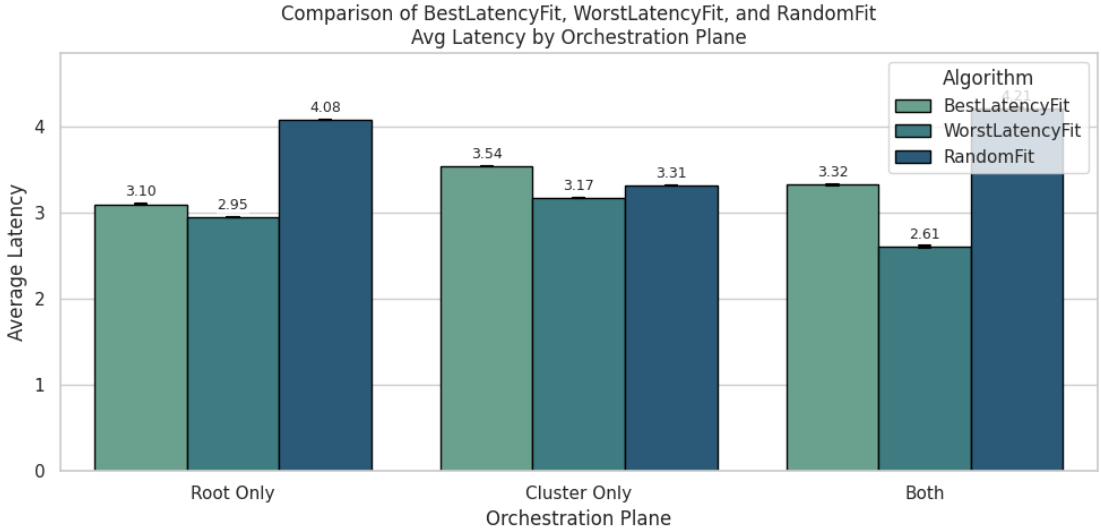

Figure 6.5: Comparison of latency-focused algorithms.

Figure 6.5 shows the resulting average latency of interdependent services with respect to the deployed scheduling algorithm. The WorstLatencyFit scheduling algorithm appears to consistently outperform BestLatencyFit. When applied at the root scheduler or at both layers, both algorithms clearly outperform the control, RandomFit. However, at the cluster orchestration plane this margin almost entirely vanishes, where only WorstLatencyFit appears to slightly outperform RandomFit. This can mostly be attributed to the latency modeling, where the inter-cluster latencies are extremely high, whereas the inter-node latencies within a cluster are comparatively low.

We conclude that the WorstLatencyFit algorithm is superior, yielding a 12% latency reduction on average, and most impactful when deployed at the root.

WorstMemoryFit and BestMemoryFit

In Figure 6.6 the resource distribution effectiveness of the scheduling algorithms is visualized. This is measured by calculating the distribution index according to Jain's fairness index, where a value close to 1 indicates an even distribution, and a value close to 0 means the services are heavily concentrated on very few nodes. Overall, BestMemoryFit performs 32% better than WorstMemoryFit, performing significantly better in evenly distributing the service load across all nodes.

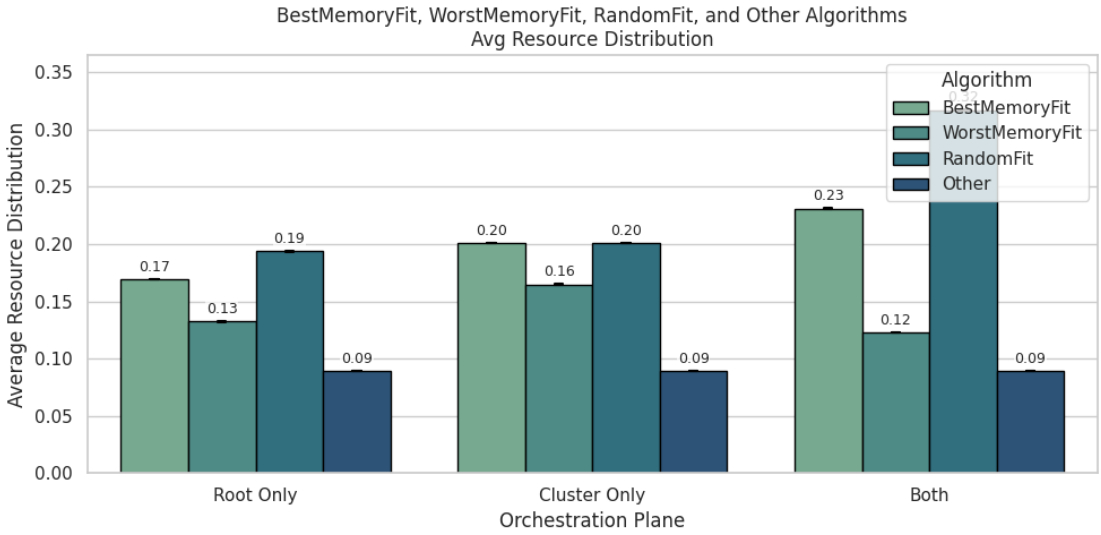

Figure 6.6: Comparison of memory-focused algorithms in terms of resource distribution.

Figure 6.7: Comparison of memory-focused algorithms in terms of error rate.

Both algorithms are, however, superseded by RandomFit. In comparison to BestMemoryFit, RandomFit only performs slightly better. We therefore use the average result of all other algorithms as a control, revealing that all three algorithms yield significant results in terms of resource distribution.

While BestMemoryFit performs better than WorstMemoryFit in terms of service distribution, Figure 6.7 shows it also incurs substantially higher scheduling error rates. A scheduling error occurs when the scheduling algorithm can no longer find a candidate that meets the filtering stage requirements. RandomFit performs well on both categories.

We observe a trade-off between BestMemoryFit and WorstMemoryFit, as the former yields a better resource distribution, but the latter incurs fewer scheduling errors. RandomFit is a viable scheduling algorithm with respect to both objectives at both orchestration planes.

LowestCarbonFit

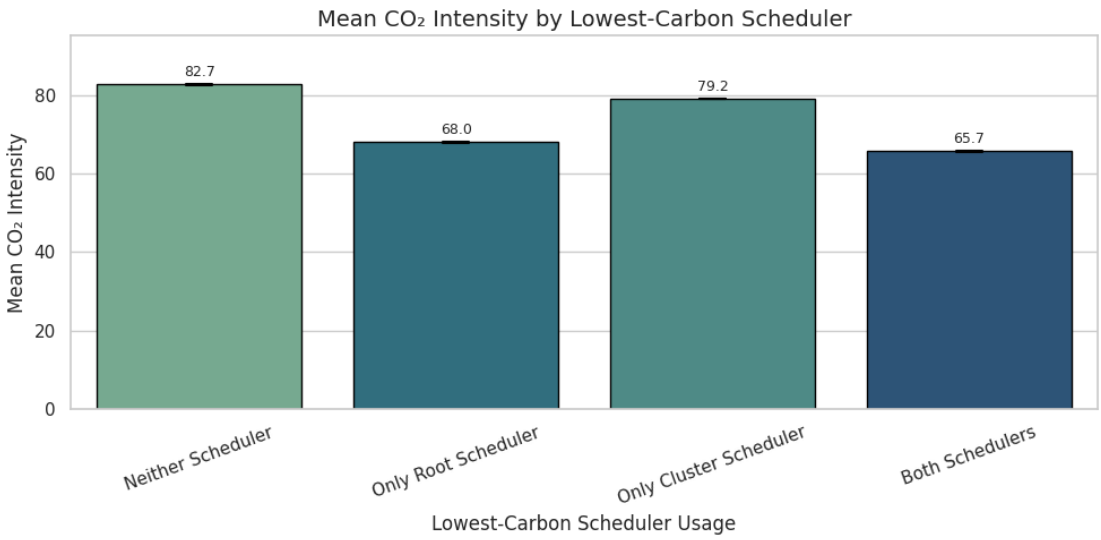

Figure 6.8: Carbon intensity by scheduling algorithm deployment.

Figure 6.8 shows the mean carbon intensity across all nodes with respect to at which orchestration plane the scheduling algorithm LowestCarbonFit was deployed. When deployed at the root, the yielded carbon intensity reduction is significant, at the cluster the effect is less pronounced. This is most likely due to the modeling of carbon intensity, where the node carbon intensity is varied by, at most, $\pm 15\%$.

We conclude that this scheduling algorithm is best deployed at the root.

6.2.3 Hierarchical Permutations Evaluation

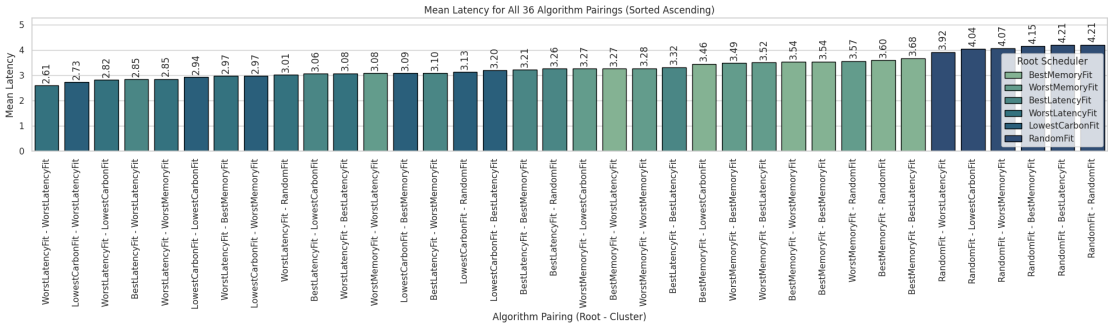

Figure 6.9: Latency by scheduling algorithm permutation.

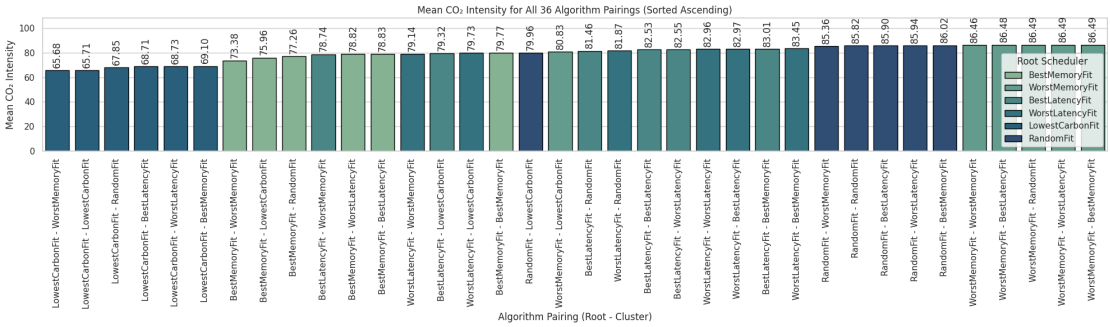

Figure 6.10: Carbon intensity by scheduling algorithm permutation.

Figure 6.9, Figure 6.10, Figure 6.11, Figure 6.12, and Figure 6.13 show all scheduling algorithm permutations and their average performance in terms of latency, carbon intensity, scheduling time, error count, and resource distribution index respectively. In most cases, the best performing scheduling algorithm permutation with respect to a single objective is, as expected, the pairing of the same algorithm that performs best concerning said objective at each orchestration plane. For example, if the only objective were to minimize latency, the WorstLatencyFit-WorstLatencyFit pairing would be the optimal permutation.

Furthermore, deploying the scheduling algorithm that best optimizes for the primary objective at the root is generally slightly more effective than at the cluster, enabling the prioritization of objectives.

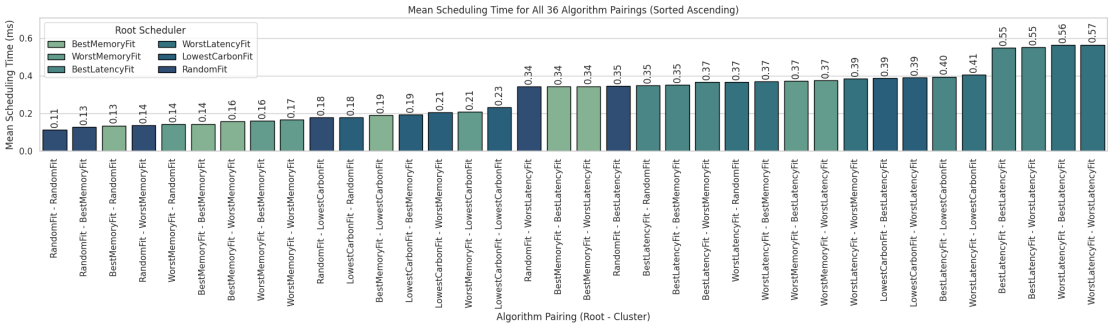

Figure 6.11: Time required by scheduling algorithm permutation.

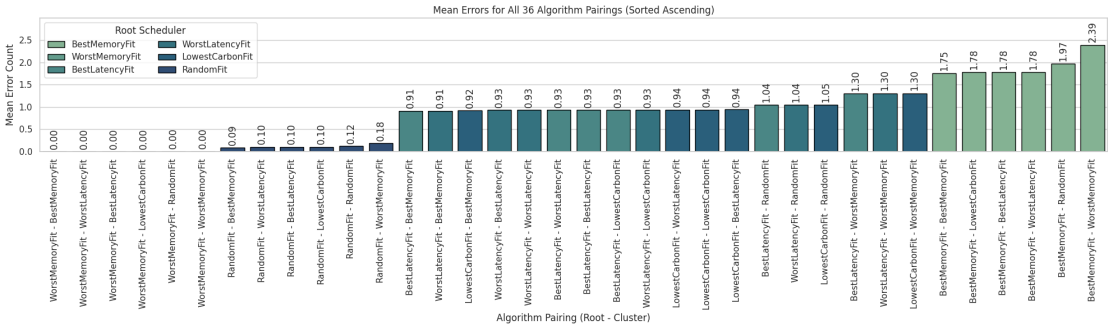

Figure 6.12: Errors incurred by scheduling algorithm permutation.

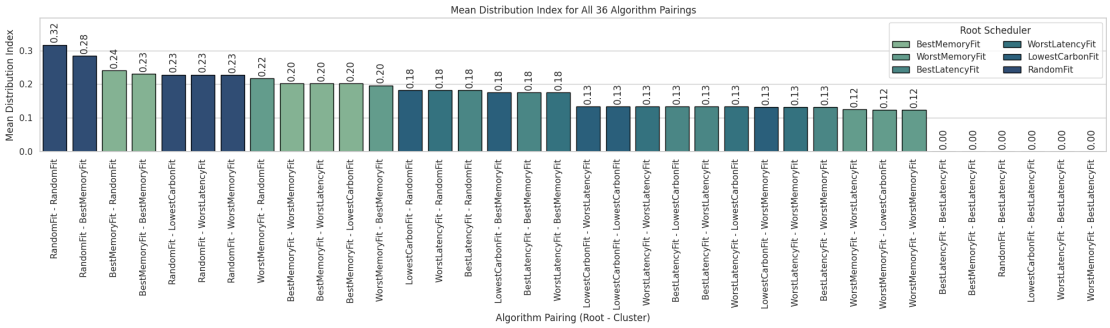

Figure 6.13: Distribution index by scheduling algorithm permutation.

Notably, deploying the BestLatencyFit algorithm at the cluster orchestration plane always yields a distribution index of 0, indicating that all services are scheduled to the same node or not scheduled at all.

Using the collected data, we rank every algorithmic permutation according to each scheduling objective. This enables an overall ranking, using the average ranking across all scheduling objectives, shown in Figure 6.14. The LowestCarbonFit-BestMemoryFit pairing achieves the highest score. Furthermore, the three highest scoring algorithmic pairings all utilize LowestCarbonFit at the root orchestration plane.

This can largely be explained by the fact that this scheduling algorithm will have the side effect of concentrating many services on a small number of nodes with a low carbon intensity. This concentration reduces the average S2S latency. Algorithmic permutations including LowestCarbonFit perform poorly in terms of distribution.

The S2S latency is arguably more significant, but the S2U latency should not be disregarded. We study the distribution index as a representative, but imprecise approximation for S2U latency. In the case that the infrastructure provider is solely interested in the S2S latency, and disregards the distributivity of the services, Figure 6.15 shows the overall score excluding the distribution index.

We conclude that the LowestCarbonFit-BestMemoryFit pairing yields the best overall results. However, LowestCarbonFit-WorstLatencyFit yields competitive results if latency is prioritized. RandomFit-BestMemoryFit and RandomFit-RandomFit both rank highly overall and are suitable alternatives if service distribution is important to the infrastructure provider.

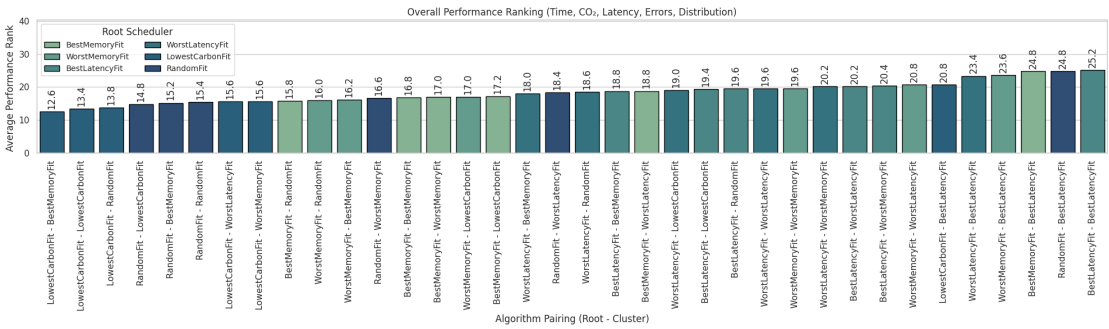

Figure 6.14: Scheduling algorithm permutation scores.

The implemented benchmarking system is accessible at a public archived repository [53].

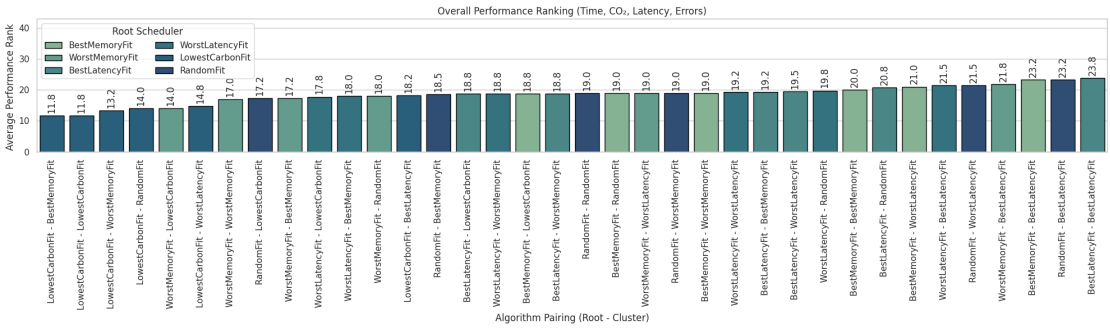

Figure 6.15: Scheduling algorithm permutation scores excluding the distribution index.

7 Conclusion

7.1 Summary

We extended the resource abstractor to the cluster orchestration plane, de-duplicating functionality and increasing the maintainability of the Oakestra framework. An aggregation scheme is introduced, allowing the agnostic collection and propagation of worker resources. These abstractions enable an abstracted scheduling component. This component can be deployed at the root and cluster orchestration plane and is capable of hosting a diverse array of scheduling algorithms.

These scheduling algorithms may consider resources the Oakestra framework has previously ignored, as the scheduler forwards the requested and required resources considered during the scheduling process without any assumptions regarding the structure of the placement candidates or services. This is achieved by treating services and candidates as two interpretations of the same dataset.

The implemented scheduler is flexible, maintainable, and performant. It enables the deployment of different scheduling layers at different orchestration planes, enabling multi-objective optimization with mono-objective, and therefore computationally simple, scheduling algorithms. The hierarchical architecture enables the even grouping of workers into clusters, significantly reducing the search space of each scheduling algorithm.

Six scheduling algorithms are implemented, addressing the objectives of latency minimization, CO2 emissions reduction, resource utilization, scheduling speed, and the minimization of SLA violations.

The scheduling algorithms are evaluated in an extensive simulation regarding their efficiency and efficacy with respect to the above objectives. In particular, the hierarchical permutations of the above scheduling algorithms are analyzed and scored according to all optimization objectives.

7.2 Limitations

The most significant limitation of this work is collecting the appropriate resources. The latency-aware scheduling algorithms use a complete symmetric matrix to resolve all

inter-candidate latencies. These must be constructed and reported by the nodes and clusters, demanding extensive measurements. Furthermore, the algorithms assume that all latency relationships are symmetric, which cannot be guaranteed across vast networks. A further given resource is the carbon intensity, which must be reported by each worker node.

The modeling of the carbon intensity assumes a strong correlation between the clustering of nodes and their relative carbon intensity. It is entirely feasible that some nodes within a cluster are connected to an entirely renewable source independent of the regional electrical grid, which may use a significantly more carbon intensive electricity mix. A further assumption made during modeling is that nodes are optimally clustered. A geographical density characterized by a lower average latency and similar carbon intensity is expected of the nodes within a cluster, while clusters are expected to be geographically distributed.

Finally, the implemented scheduling algorithms are not optimal. Superior algorithms in terms of efficiency or efficacy can be found in the literature.

7.3 Future Work

We consider the most immediate challenge to be the implementation of resource collection systems into the Oakestra framework, as this is an outstanding requirement for real-world integration.

The abstracted scheduler offers many possibilities in terms of scheduling flexibility and optimizations. Further research is required to establish optimal scheduling algorithms and their permutations. Additionally, further objectives such as privacy and legal compliance are deserving of attention.

The assumptions surrounding the logical clustering of nodes are not entirely unfounded, as the infrastructure provider is incentivized to construct the architecture in a rational manner. However, automated approaches for node clustering are feasible, and could provide dynamic solutions.

Finally, the current scheduler implementation enables the flexible deployment of scheduling algorithms at each orchestration plane. However, this flexibility is not dynamic. Future iterations could address this limitation by allowing the exchanging of scheduling algorithms at runtime.

Abbreviations

IoT Internet of Things

CRAS Connected Robotics and Autonomous Systems

CAV Connected Autonomous Vehicles

SPP Service Placement Problem

RA Resource Abstractor

RO Root Orchestrator

CO Cluster Orchestrator

vCPU Virtual Central Processing Unit

SLA Service Level Agreement

S2S Service-to-Service

S2U Service-to-User

UAV Unmanned Aerial Vehicle

ILP Integer Linear Programming

MILP Mixed-Integer Linear Programming

NCS Network Coordinate System

List of Figures

2.1 An example SLA [10]. 5
2.2 Architecture of the root orchestrator [10]. 9
2.3 Architecture of the cluster orchestrator [10]. 10
2.4 Architecture of the worker node [10]. 11

4.1 Rearchitecting the orchestration plane. 14
4.2 Architecture of the scheduling component. 18
4.3 Example `ResourceList` and `Algorithm` implementation. 19

5.1 Latency-aware scheduling example. 23
5.2 Latency-aware candidate filtering process. 24

6.1 `LatencyAwareResources` used for benchmarking. 26
6.2 Benchmarking node distribution. 27
6.3 Scheduling time by cluster-node configuration. 30
6.4 Computational intensity of scheduling algorithms. 31
6.5 Comparison of latency-focused algorithms. 32
6.6 Comparison of memory-focused algorithms in terms of resource distribution. 33
6.7 Comparison of memory-focused algorithms in terms of error rate. . . . 33
6.8 Carbon intensity by scheduling algorithm deployment. 34
6.9 Latency by scheduling algorithm permutation. 35
6.10 Carbon intensity by scheduling algorithm permutation. 35
6.11 Time required by scheduling algorithm permutation. 36
6.12 Errors incurred by scheduling algorithm permutation. 36
6.13 Distribution index by scheduling algorithm permutation. 36
6.14 Scheduling algorithm permutation scores. 37
6.15 Scheduling algorithm permutation scores excluding the distribution index. 38

List of Tables

4.1 Canonical and Non-canonical Resources 15
4.2 Abstract Aggregation Scheme 17

5.1 latency categories 21

6.1 Inter-node Latency Distribution 27
6.2 Inter-cluster Latency Distribution 28
6.3 Service Resource Requirements 28

Bibliography

[1] T. I. Research. *Global IoT market to grow to 24.1 billion devices in 2030, generating $1.5 trillion annual revenue*. https://transformainsights.com/news/iot-market-24-billion-usd15-trillion-revenue-2030. 2020.

[2] A. Hazra, P. Rana, M. Adhikari, and T. Amgoth. "Fog computing for next-generation Internet of Things: Fundamental, state-of-the-art and research challenges." In: *Computer Science Review* 48 (2023), p. 100549. ISSN: 1574-0137. DOI: https://doi.org/10.1016/j.cosrev.2023.100549.

[3] C. D. Alwis, A. Kalla, Q.-V. Pham, P. Kumar, K. Dev, W.-J. Hwang, and M. Liyanage. "Survey on 6G Frontiers: Trends, Applications, Requirements, Technologies and Future Research." In: *IEEE Open Journal of the Communications Society* 2 (2021), pp. 836–886. DOI: 10.1109/OJCOMS.2021.3071496.

[4] W. Saad, M. Bennis, and M. Chen. "A Vision of 6G Wireless Systems: Applications, Trends, Technologies, and Open Research Problems." In: *IEEE Network* 34.3 (2020), pp. 134–142. DOI: 10.1109/MNET.001.1900287.

[5] V. Cozzolino, L. Tonetto, N. Mohan, A. Y. Ding, and J. Ott. "Nimbus: Towards Latency-Energy Efficient Task Offloading for AR Services." In: *IEEE Transactions on Cloud Computing* 11.2 (2023), pp. 1530–1545. DOI: 10.1109/TCC.2022.3146615.

[6] A. AlZailaa, H. R. Chi, A. Radwan, and R. Aguiar. "Low-Latency Task Classification and Scheduling in Fog/Cloud based Critical e-Health Applications." In: *ICC 2021 - IEEE International Conference on Communications*. 2021, pp. 1–6. DOI: 10.1109/ICC42927.2021.9500985.

[7] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue. "All one needs to know about fog computing and related edge computing paradigms: A complete survey." In: *Journal of Systems Architecture* 98 (2019), pp. 289–330. ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2019.02.009.

[8] S. A. Noghabi, L. Cox, S. Agarwal, and G. Ananthanarayanan. "The Emerging Landscape of Edge Computing." In: *GetMobile: Mobile Comp. and Comm.* 23.4 (May 2020), pp. 11–20. ISSN: 2375-0529. DOI: 10.1145/3400713.3400717.

[9] W. Shi and S. Dustdar. "The Promise of Edge Computing." In: *Computer* 49.5 (2016), pp. 78–81. DOI: 10.1109/MC.2016.145.

[10] Oakestra. *Oakestra Docs*. https://www.oakestra.io/docs/. 2025.

[11] F. A. Salaht, F. Desprez, and A. Lebre. "An Overview of Service Placement Problem in Fog and Edge Computing." In: *ACM Comput. Surv.* 53.3 (June 2020). ISSN: 0360-0300. DOI: 10.1145/3391196.

[12] R. Mahmud, S. N. Srirama, K. Ramamohanarao, and R. Buyya. "Quality of Experience (QoE)-aware placement of applications in Fog computing environments." In: *Journal of Parallel and Distributed Computing* 132 (2019), pp. 190–203. ISSN: 0743-7315. DOI: https://doi.org/10.1016/j.jpdc.2018.03.004.

[13] V. Scoca, A. Aral, I. Brandic, R. De Nicola, and R. B. Uriarte. "Scheduling Latency-Sensitive Applications in Edge Computing." In: Jan. 2018, pp. 158–168. DOI: 10.5220/0006706201580168.

[14] H. Tan, Z. Han, X.-Y. Li, and F. C. Lau. "Online job dispatching and scheduling in edge-clouds." In: *IEEE INFOCOM 2017 - IEEE Conference on Computer Communications*. 2017, pp. 1–9. DOI: 10.1109/INFOCOM.2017.8057116.

[15] V. B. C. Souza, W. Ramírez, X. Masip-Bruin, E. Marín-Tordera, G. Ren, and G. Tashakor. "Handling service allocation in combined Fog-cloud scenarios." In: *2016 IEEE International Conference on Communications (ICC)*. 2016, pp. 1–5. DOI: 10.1109/ICC.2016.7511465.

[16] M. Taneja and A. Davy. "Resource aware placement of IoT application modules in Fog-Cloud Computing Paradigm." In: *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*. 2017, pp. 1222–1228. DOI: 10.23919/INM.2017.7987464.

[17] A. Alsaffar, P. Hung, C. S. Hong, E.-n. Huh, and M. Aazam. "An Architecture of IoT Service Delegation and Resource Allocation Based on Collaboration between Fog and Cloud Computing." In: *Mobile Information Systems* 2016 (Jan. 2016), pp. 1–15. DOI: 10.1155/2016/6123234.

[18] G. Premsankar and B. Ghaddar. "Energy-Efficient Service Placement for Latency-Sensitive Applications in Edge Computing." In: *IEEE Internet of Things Journal* 9.18 (2022), pp. 17926–17937. DOI: 10.1109/JIOT.2022.3162581.

[19] S. Rac and M. Brorsson. "Cost-aware Service Placement and Scheduling in the Edge-Cloud Continuum." In: *ACM Trans. Archit. Code Optim.* 21.2 (Mar. 2024). ISSN: 1544-3566. DOI: 10.1145/3640823.

[20] N. Asadov, V. C. Coroamă, M. Franzil, S. Galantino, and M. Finkbeiner. "Carbon-Aware Spatio-Temporal Workload Shifting in Edge–Cloud Environments: A Review and Novel Algorithm." In: *Sustainability* 17.14 (2025). ISSN: 2071-1050. DOI: 10.3390/su17146433.

[21] J. Santos, T. Wauters, B. Volckaert, and F. De Turck. "Resource provisioning for IoT application services in smart cities." In: *2017 13th International Conference on Network and Service Management (CNSM)*. 2017, pp. 1–9. DOI: 10.23919/CNSM.2017.8255974.

[22] C. T. Do, N. H. Tran, C. Pham, M. G. R. Alam, J. H. Son, and C. S. Hong. "A proximal algorithm for joint resource allocation and minimizing carbon footprint in geo-distributed fog computing." In: *2015 International Conference on Information Networking (ICOIN)*. 2015, pp. 324–329. DOI: 10.1109/ICOIN.2015.7057905.

[23] M. Trabelsi, N. M. Mehdi Bendaoud, and S. B. Ahmed. "Multi-objective Optimization for Dynamic Service Placement Strategy for Real-Time Applications in Fog Infrastructure." In: *2023 IEEE Symposium on Computers and Communications (ISCC)*. 2023, pp. 607–612. DOI: 10.1109/ISCC58397.2023.10217950.

[24] C. Guo, L. Yuan, D. Xiang, Y. Dang, R. Huang, D. Maltz, Z. Liu, V. Wang, B. Pang, H. Chen, Z.-W. Lin, and V. Kurien. "Pingmesh: A Large-Scale System for Data Center Network Latency Measurement and Analysis." In: *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. SIGCOMM '15. London, United Kingdom: Association for Computing Machinery, 2015, pp. 139–152. ISBN: 9781450335423. DOI: 10.1145/2785956.2787496.

[25] A. Schmidt, G. Stock, R. Ohs, L. Gerhorst, B. Herzog, and T. Hönig. "carbond: An Operating-System Daemon for Carbon Awareness." In: *Proceedings of the 2nd Workshop on Sustainable Computer Systems*. HotCarbon '23. Boston, MA, USA: Association for Computing Machinery, 2023. ISBN: 9798400702426. DOI: 10.1145/3604930.3605707.

[26] Electricity Maps ApS. *Electricity Maps API Documentation*. https://app.electricitymaps.com/developer-hub/api/getting-started. Accessed: 2025-11-30. 2025.

[27] T. linux foundation. *Kubernetes*. 2025.

[28] C. N. C. Foundation. *Digital transformation driven by community: Kubernetes as example*. https://www.cncf.io/blog/2025/01/30/digital-transformation-driven-by-community-kubernetes-as-example/. 2025.

[29] A. Jeffery, H. Howard, and R. Mortier. "Rearchitecting Kubernetes for the Edge." In: *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking*. EdgeSys '21. Online, United Kingdom: Association for Computing Machinery, 2021, pp. 7–12. ISBN: 9781450382915. DOI: 10.1145/3434770.3459730.

[30] S. Sari. "Survey of Container Orchestration Distributions in Cloud-to-Edge Continuum." In: *2025 12th International Conference on Future Internet of Things and Cloud (FiCloud)*. 2025, pp. 39–45. DOI: 10.1109/FiCloud66139.2025.00014.

[31] G. Bartolomeo, M. Yosofie, S. Bäurle, O. Haluszczynski, N. Mohan, and J. Ott. "Oakestra: A Lightweight Hierarchical Orchestration Framework for Edge Computing." In: *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, July 2023, pp. 215–231. ISBN: 978-1-939133-35-9.

[32] Docker. *Docker Docs: Networking in Compose*. https://docs.docker.com/compose/how-tos/networking/. 2025.

[33] Go. *The Go Programming Language Specification*. https://go.dev/ref/spec. 2025.

[34] Python. *Python Glossary*. https://docs.python.org/3/glossary.html. 2025.

[35] M. Inc. *MongoDB: The Developer Data Platform*. https://www.mongodb.com/. Accessed: 2025-11-12. 2025.

[36] M. Elkodary. "Enhancing Edge Orchestration Flexibility through Addons." PhD thesis. Master's thesis, Technical University of Munich, 2024. Accessed: 2025-11-25.

[37] O. Standard. *MQTT Version 5.0*. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html. OASIS Standard. 2019.

[38] T. G. Authors. *The Go Programming Language*. https://go.dev/. Version 1.24. 2025.

[39] T. B. G. Team. *Measured: "Which programming language is fastest?"* https://benchmarksgame-team.pages.debian.net/benchmarksgame/. Accessed: 2025-11-12. 2025.

[40] Hibiken. *Asynq: Simple, reliable & efficient distributed task queue in Go*. https://github.com/hibiken/asynq. Version 0.25.1. 2025.

[41] R. Ltd. *Redis: The Real-time Data Platform*. https://https://redis.io/. 2025.

[42] P. Di Francesco, P. Lago, and I. Malavolta. "Migrating Towards Microservice Architectures: An Industrial Survey." In: *2018 IEEE International Conference on Software Architecture (ICSA)*. 2018, pp. 29–2909. DOI: 10.1109/ICSA.2018.00012.

[43] A. Networks. *Application Delivery Service Challenges in Microservices-Based Applications*. https://www.a10networks.com/blog/application-delivery-service-challenges-microservices-based-applications/. 2016.

[44] T. Høiland-Jørgensen, B. Ahlgren, P. Hurtig, and A. Brunstrom. "Measuring Latency Variation in the Internet." In: Nov. 2016, pp. 473–480. DOI: 10.1145/2999572.2999603.

[45] M. Yang, J. Zhang, and L. Yu. "Perceptual Tolerance to Motion-To-Photon Latency with Head Movement in Virtual Reality." In: *2019 Picture Coding Symposium (PCS)*. 2019, pp. 1–5. DOI: 10.1109/PCS48520.2019.8954518.

[46] Y. Takahashi, K. Hakamada, H. Morohashi, H. Akasaka, Y. Ebihara, E. Oki, S. Hirano, and M. Mori. "Verification of delay time and image compression thresholds for telesurgery." In: *Asian journal of endoscopic surgery* 16 (Dec. 2022). DOI: 10.1111/ases.13150.

[47] K. Raaen, R. Eg, and C. Griwodz. "Can gamers detect cloud delay?" In: *2014 13th Annual Workshop on Network and Systems Support for Games*. 2014, pp. 1–3. DOI: 10.1109/NetGames.2014.7008962.

[48] N. González, M. Solera, F. Ruiz, C. Gijón, and M. Toril. "A quality of experience model for live video in first-person-view drone control in cellular networks." In: *Computer Networks* 237 (2023), p. 110089. ISSN: 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2023.110089.

[49] J. D. Brutlag, H. Hutchinson, and M. Stone. "User Preference and Search Engine Latency." In: (2008).

[50] M. Humphreys. *Oakestra Thesis Fork*. https://github.com/Mjaethers/oakestra-bachelor-thesis. Accessed: 2025-11-28. 2025.

[51] B. Whitehead, D. Andrews, and A. Shah. "The life cycle assessment of a UK data centre." In: *The International Journal of Life Cycle Assessment* 20 (Mar. 2015). DOI: 10.1007/s11367-014-0838-7.

[52] G. Foundation. *Carbon Aware SKD*. https://carbon-aware-sdk.greensoftware.foundation/. Accessed: 2025-11-24. 2025.

[53] M. Humphreys. *Oakestra Thesis Fork - Evaluation*. https://github.com/Mjaethers/oakestra-bachelor-thesis/tree/scheduling-algorithms-evaluation. Accessed: 2025-11-28. 2025.